

Playwright with TypeScript

Week 5 – Home Assignment



Program	Playwright with TypeScript
Module	TypeScript OOP Concepts
Week	Week 5
Assignment	Interface and Abstract Class (Automation Testing Context)

1. Objective

To practice and understand the difference between interface and abstract class in TypeScript by implementing them using a real-time automation testing scenario.

2. Problem Statement

Create a simple TypeScript program for a mini page action framework using both interface and abstract class. Define a common contract using an interface and provide shared functionality using an abstract class. Implement concrete page classes and use them to perform basic page actions.

3. Requirements

- Create an interface called **IPageActions** with the following methods:
 - open(url: string): void
 - getTitle(): string
 - isElementVisible(locator: string): boolean
- Create an abstract class called **BasePage** that implements **IPageActions** with the following:
 - Property: pageName (string)
 - Constructor to set the pageName
 - Concrete method: logPageInfo() to print the page name
 - Abstract methods: open(url: string): void, getTitle(): string, isElementVisible(locator: string): boolean
- Create a child class called **LoginPage** that extends **BasePage** and implements the abstract methods:
 - open(url: string): void → print a message (e.g., Navigating to login page: <url>)
 - getTitle(): string → return a sample title (e.g., "Login Page - OrangeHRM")
 - isElementVisible(locator: string): boolean → return true and print a message
 - Additional method: login(username: string, password: string): void to simulate login action
- Create another child class called **DashboardPage** that extends **BasePage** and implements the abstract methods:
 - open(url: string): void → print a message (e.g., Navigating to dashboard page: <url>)
 - getTitle(): string → return a sample title (e.g., "Dashboard - OrangeHRM")
 - isElementVisible(locator: string): boolean → return true and print a message
 - Additional method: verifyWelcomeMessage(): void to simulate welcome message validation
- Create objects for both LoginPage and DashboardPage and call their methods to demonstrate the use of interface and abstract class.
- Add meaningful comments and use proper TypeScript types.

4. Sample Test Data and Expected Output

Sample Test Data (Code)	Expected Output (Console)
<pre>// Create LoginPage object const loginPage = new LoginPage("LoginPage"); loginPage.open("https://opensource-demo.orangehrmlive.com/"); console.log(loginPage.getTitle()); loginPage.isElementVisible("#username"); loginPage.login("Admin", "admin123"); // Create DashboardPage object const dashboardPage = new DashboardPage("DashboardPage"); dashboardPage.open("https://opensource-demo.orangehrmlive.com/"); console.log(dashboardPage.getTitle()); dashboardPage.isElementVisible("#welcome"); dashboardPage.verifyWelcomeMessage();</pre>	<pre>Page: LoginPage Navigating to login page: https://opensource-demo.orangehrmlive.com/ Title: Login Page - OrangeHRM Element #username is visible: true Performing login with username: Admin, password: admin123 Page: DashboardPage Navigating to dashboard page: https://opensource-demo.orangehrmlive.com/ Title: Dashboard - OrangeHRM Element #welcome is visible: true Dashboard welcome message is displayed successfully.</pre>

5. Technical Guidelines

- Use TypeScript (.ts file).
- Create an interface and define method signatures.
- Create an abstract class and implement the interface.
- Implement abstract methods in child classes using the override keyword.
- Use meaningful method and class names related to automation testing.
- Use console.log() to display the output.
- Add comments for better understanding.
- Follow proper naming conventions and code formatting.

6. Learning Outcome

After completing this assignment, you should be able to:

- Understand and use interface in TypeScript.
- Understand and use abstract class in TypeScript.
- Identify the differences and use cases between interface and abstract class.
- Apply these concepts in a real-time automation testing scenario.